Memory system

Computer system contains a variety of devices to store instruction and data for its operation

Such storage devices along with the algorithms needed to control or manage the stored information constitute the memory system of the computer

Time required to transfer information between processor and memory should be such that processor can work at high speed

Memories that operate at speeds comparable to processor speeds are costly. Hence it is not feasible to employ a single memory using just one type of technology. Instead the stored information is distributed in complex fashion over a variety of different memory units with very different physical characteristics

Memory components of a computer system can be divided into 3 main groups:

1. Internal processor memory: A small set of high-speed registers used as a working memory for temporary storage of instruction and data
2. Main memory (also called primary memory): A relatively large fast memory used for program and data storage during computer operation. It is characterized by the fact that locations in main memory can be accessed directly and rapidly by the CPU instruction set. The principal technology used for main memory is based on semiconductor integrated circuits (ICs).
3. Semiconductor memory (also called auxiliary or backing memory): Much larger in capacity and also much slower than main memory. Used to store system programs, large data files, which are not continually required by CPU; also serves as an overflow memory when the capacity of the main memory is exceeded. Information in secondary storage is accessed indirectly via input-output programs that first transfer the required information to main memory. Representative technologies used for secondary memory are magnetic tapes and disks.

An increasing number of machines employ another type of memory called a cache, which serves as an intermediate temporary storage unit logically positioned between the processor registers and main memory.

Major objective in designing any memory system is to provide adequate storage capacity with an acceptable level of performance at a reasonable cost. Four important interrelated ways of approaching this goal can be identified.

1. Use of number of different memory devices with different cost/performance ratios organized to provide a high average performance at a low average cost/bit. The individual memories form a hierarchy of storage devices.
2. Development of automatic storage allocation methods to make more efficient use of the available memory space.
3. Development of virtual memory concepts to free the ordinary user from memory management, and make programs largely independent of the physical memory configurations used
4. Design of communication links to the memory system so that all processors connected to it can operate at or near their maximum rates. This involves increasing the effective memory processor bandwidth and also providing protection mechanisms to prevent programs from accessing or altering one another's storage areas.

Memory-Device characteristics

Cost: Purchase price include cost of the information storage cells, cost of the peripheral equipment or access circuitry essential to the operation of the memory

C = Cost of a complete memory system with S bits of storage capacity, c= cost/bit = C/S

Access time: Performance of a memory device is primarily determined by the rate at which information can be read from or written into the memory. Read (Write) access time is the average time required to read (write) a fixed amount of information from (into) the memory. It depends on the physical characteristic of the storage medium and the type of access mechanism. Access time (t_A) is calculated from the time a read (write) request is received by the memory unit to the time at which all the requested information has been made available at the memory output terminals (written into the memory).

Access rate (b_A): Defined as $1/t_A$ and measured in words per second.

Access modes: An important property of a memory device is the order or sequence in which information can be accessed.

RAM:
If locations may be accessed in any order and access time is independent of the location being accessed, the memory is termed a random accessed memory (RAM).
Ferrite-core and semiconductor memories are usually of this type.
Each storage location can be accessed independently of the other locations and hence needs a separate access mechanism, or read/write head for every location.

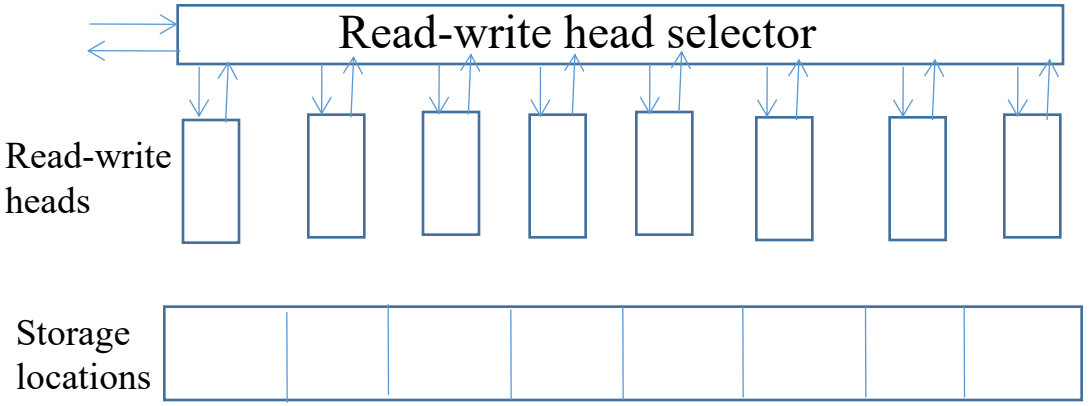


Fig. Conceptual model of RAM

SAM:
Memories where storage locations can be accessed only in certain predetermined sequences.
Magnetic-tape, magnetic-bubble, optical memories employ serial access methods.
The access mechanism is shared among different locations. It must be assigned to different locations at different times. This is accomplished by moving the stored information, the read/write head or both.
Many serial access memories operate by continually moving the storage locations around a closed path or track. A particular location can be accessed only when it passes the fixed read/write head; thus the time required to access a particular location depends on its position relative to the read/write head when the access request is received.

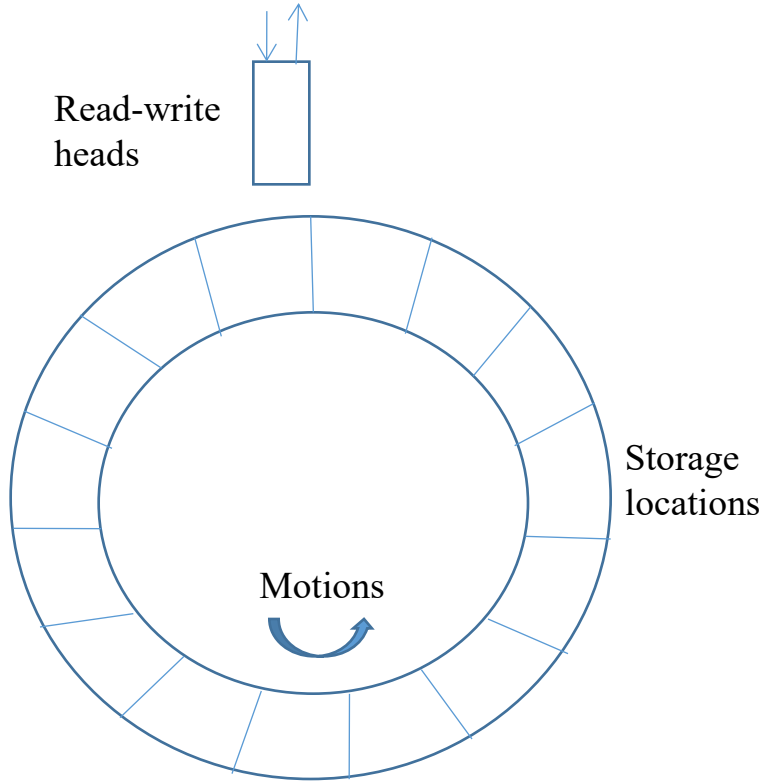


Fig. Conceptual model of SAM

RAM vs. SAM:
Since every location has its own addressing mechanism, RAMs tend to be more costly than SAM
In SAM, the time required to bring the desired location into correspondence with a read/write head increases the effective access time. So SAM is slower than RAM.

Thus the access mode employed contributes significantly to the inverse relation between cost and access time.

Semi-random access mode:
Some memory devices such as magnetic disks contain a large number of independent rotating tracks. If each track has its own read/write head, the tracks may be accessed randomly, although the access within each track is serial.

Alterability; ROMs:
Once the information has been written, it can not be altered while the memory is in use.
Nonerasable storage device.
Widely used for storing control programs such as microprograms.
ROMs whose content can be changed are called programmable ROMs (PROMs).

Read-write memories:
Memories in which reading or writing can be done are sometimes called read-write memories.
All memories used for temporary storage purposes are read-write memories.

Permanence of storage:
Physical processes involved in storage are sometimes inherently unstable, so that the stored information may be lost over a period of time unless appropriate action is taken.
Three different memory characteristics that can destroy information:
Destructive read out (DRO)
Method of reading the memory destroys the stored information.
Each read operation must be followed by a write operation that restores the original state of the memory. This restoration is usually carried out automatically using a buffer register. The word at the addressed location is transferred to the buffer register where it is available to external devices. The contents of the buffer are automatically written back into the location originally addressed.

NDRO: Memories in which reading does not affect the stored data are said to have nondestructive readout (NDRO).
Dynamic memory:
Certain memory devices have the property that a stored 1 tends to become a 0, or vice versa, due to some physical decay process.
Requires periodic refreshing to avoid the loss of information.
Memories using magnetic storage techniques are static.

Volatile:
Physical process that can destroy the contents of a memory is the failure of its power supply.
A memory is said to be volatile if the stored information can be destroyed by a power failure.
Most semiconductor memories are nonvolatile.

Cycle time:
Time needed to complete any read or write operation in the memory is the cycle time (t_M).
Data transfer rate or bandwidth ($b_M=1/t_M$):
Maximum amount of information that can be transferred to or from the memory every second.
Data transfer rate is measured in bits or words per second.

In DRO and dynamic memories, it may not be possible to initiate another memory access until a restore or refresh operation has been carried out. In such cases $t_A \neq t_M$, both are used to measure memory speed.
Access time may be more important in measuring overall computer system performance since it determines the length of time a processor must wait after initiating a memory access request; during the remainder of the memory cycle, both processor and memory can operate simultaneously.
If $t_A \neq t_{M,}$ then $b_A \neq b_M$ and b_A does not represent the actual number of accesses that can be carried out per second, whereas b_M does.

Memory system

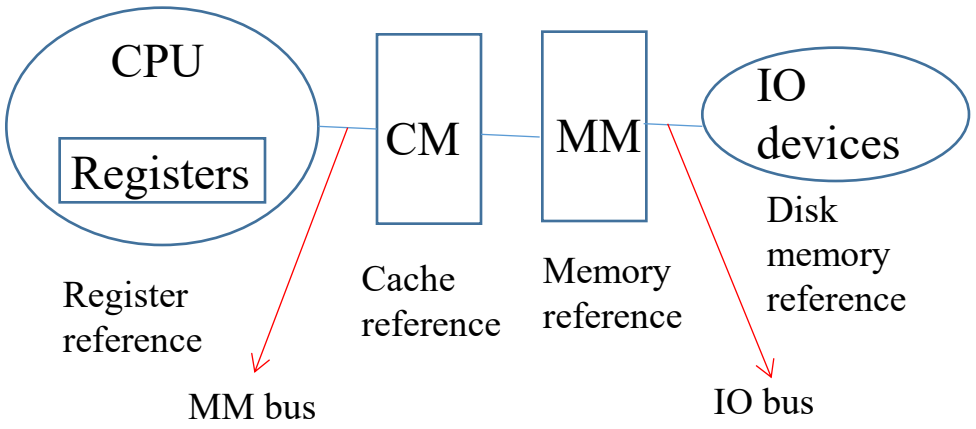
Programmers want unlimited amount of fast memory

Most of the programs do not access all code or data uniformly

Smaller hardwire is faster and fast memory is expensive

Memory hierarchy contains several levels of memory, the goal is to provide a memory system with cost almost as low as the cheapest level of memory and speed almost as fast as the fastest level

All data in N^{th} level is also available in $(N+1)^{th}$ level of the hierarchy



Cost/bit = $\frac{S1C1 + S2C2}{S1 + S2}$

As S2>>S1, Cost/bit=C2

S1 and S2 are the size of level 1 and level 2 of memory

C1 and C2 are cost/bit of level 1 and level 2 of memory

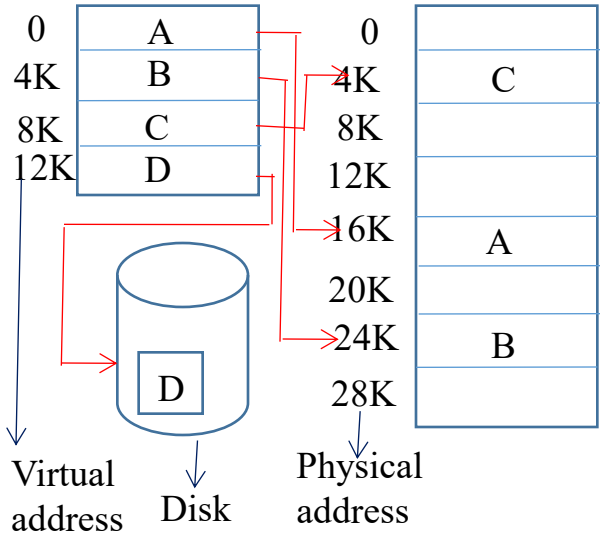
Virtual memory

Computer runs multiple processes simultaneously and each process needs its own address space. So it is required to share smaller amount of physical memory among multiple processes

A program may be too large for physical memory. Programs may be divided into pieces and loads the pieces that are required for current execution into the physical memory and kept the rest of the pieces into the virtual memory

Virtual memory reduces the burden of the programmer by automatically managing the two levels of the memory hierarchy represented by main memory and secondary memory

Mapping of virtual memory to physical memory for a program with 4 pages



Logical program is in the contiguous virtual address space. It contains 4 pages. Pages A, B and C are located in physical memory whereas page D is located in disk

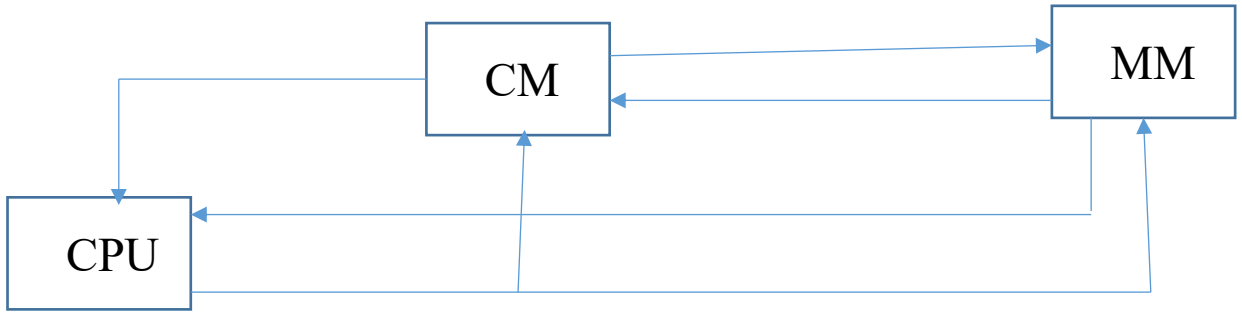
Virtual memory simplifies the loading of program for execution

Logical program can be placed any where in the physical memory or disk by changing the mapping between them

In presence of virtual memory CPU generates virtual addresses during the program execution. The virtual address is translated by a combination of hardware and software to physical address for accessing the physical memory. This process is called memory mapping or address translation.

Cache memory

First level of memory hierarchy
CPU has direct access to both cache memory (CM) and main memory (MM)
Both CM and MM are divided into equal sized units, called block in case of MM and block frame in case of CM for uniformity of reference



Number of blocks in MM (Fig 1(b)) and block frames in CM (Fig 1(a)) are assumed as 8 and 4 respectively
So at any instant of time 4 blocks of MM can be placed in 4 block frames of CM
Size of block address and block frame address are 3-bit (000 to 111 shown in Fig 1(b)) and 2-bit (00 to 11 shown in Fig 1(a)) respectively.
Size of block and block frame is assumed as 1-Kbyte i.e. each block and block frame has 1024-byte.
Size of byte address (Block offset) for accessing a byte from 1024-byte in a block is 10-bit.
The **MM address generated by CPU during program execution is assumed as 101 1001010110** i.e. the CPU generates a request for byte number 1001010110 from MM block number 101.

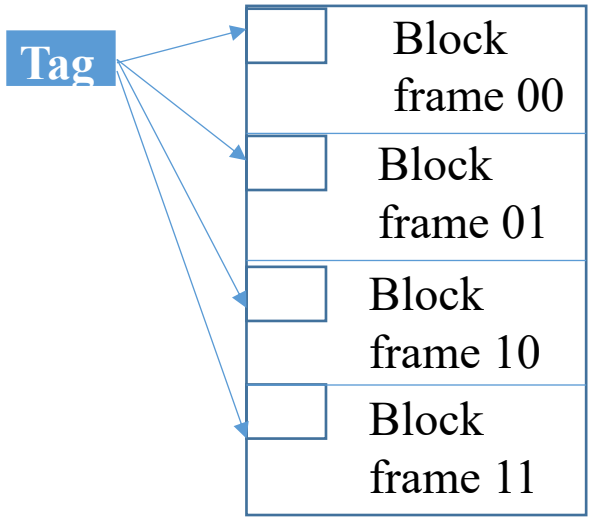


Fig 1(a) CM with 4 block frames

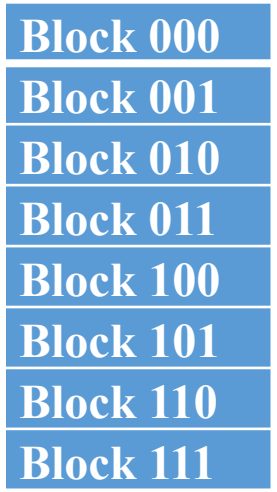


Fig 1(b) MM with 8 blocks

Each block frame in CM has an address tag.
Information in this tag is checked to find hit or miss.
Direct mapped, fully associative, set associative are the three categories of cache organization depending on the way of placing MM block in CM block frame

Direct mapped cache:

A particular block of MM can be placed in a particular block frame of CM
Direct mapping is one way set associative mapping and can be thought of as having m sets
Address in CM is computed as (MM block number) mod (Number of block frames in CM).
For example, MM block 0 is mapped into CM block frame 0 as 0 mod 4 is 0, MM block 6 is mapped into CM block frame 2 as 6 mod 4 is 2.

Fully associative cache:

A MM block can be placed in any block frame of CM i.e. anywhere in CM. If fully associative cache memory has m block frames, it is m-way set associative. It can be thought of as having a single set.

Set associative cache:

CM is divided into few sets and each set contains a few block frames.
A MM block can be placed in any block frame within a particular set. This particular set address is computed as (MM block number) mod (Number of sets in CM).
Fig 2 shows a 2-way set associative CM in which CM is divided into 2 sets (Set 0 and Set 1).
Each set has 2 block frames due to which this CM is known as 2-way set associative.
For example, MM block 0 is mapped into CM Set 0 as 0 mod 2 is 0, MM block 5 is mapped into CM Set 1 as 5 mod 2 is 1.

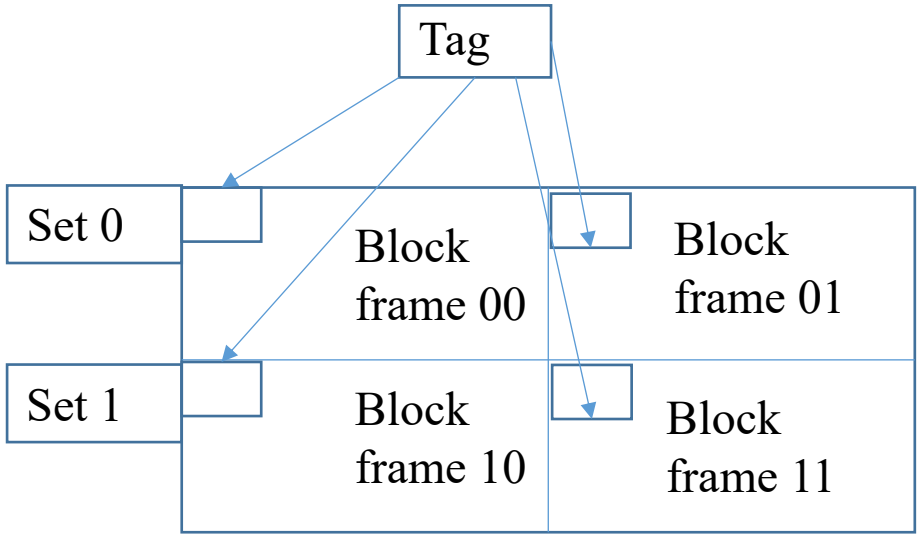


Fig 2 2-way set associative cache memory

Block replacement strategy

In case of direct mapped only one block frame is checked for a hit and only that block can be replaced

In case of fully associative or set associative there are many block frames to choose from on a miss and that can be done either randomly or using least recently used strategy (LRU)

Random replacement is simple to implement in hardwire.

LRU becomes expensive if the number of block increases.

Write: Modifying data is not possible until the tag is checked for hit. So write takes longer time than read. Processor specifies the size of write usually between 1 to 8 bytes so that only that portion of a block can be changed whereas read can access more bytes than necessary without any fear.

Read: Block can be read at the same time that the tag is read and compared. Block read begins as soon as the block address is available. If hit, data is passed on to CPU otherwise ignore read.

Write through	Write back
Information is written to both the block in the cache and to the block in the lower level memory Read misses never result in writes to the lower level Easier to implement Next lower level has the most current copy of the data CPU has to wait till write in the lower level is completed and CPU is said to write stall Write stall of CPU can be reduced by using write buffer which allows the processor to continue as soon as the data is written to the buffer, thereby overlapping processor execution with memory updating	Information is written only to the block in the cache Modified block is written to main memory when it is replaced Frequency of writing back can be reduced by using dirty bit in each block Write occurs at the speed of cache memory Multiple writes within a block requires only one write to the lower level memory Since some writes do not go to memory, write back needs less memory bandwidth, making write back attractive for multiprocessor

Actions on write miss	
Write allocate also called fetch on write	No write allocate also called write around
Block is loaded on a write miss followed by the write hit action	Block is modified in the lower level and not loaded in cache

Cache miss: Three types of cache misses are compulsory miss, conflict miss and capacity miss. First access to a memory block causes compulsory miss. If the cache can not contain all the blocks needed during execution of program, capacity misses will occur because of blocks being discarded and later retrieved. Conflict/collision/interference miss occurs if the block placement strategy is direct mapped or set associative.

Example of conflict miss and capacity miss when CPU generates requests for MM block number 0, 4, 5, 1, 3 during program execution.

Block frame address	00	01	10	11
Block frame content	~~0~~ 4	~~5~~ 1		3

Fig 2(a): CM in Fig 1(a) is considered as direct mapped and is redrawn in Fig 2(a)

- 1st CPU request is for MM block 0. CM block frame address is computed as 0 mod 4 =0. So CM block frame 00 is now occupied by MM block 0.
- 2nd CPU request is for MM block 4. CM block frame address is computed as 4 mod 4 =0. But the CM block frame 00 is already occupied by MM block 0. **Conflict miss** occurs **between MM block 0 and MM block 4.**
- 3rd CPU request is for MM block 5. CM block frame address is computed as 5 mod 4 =1. So CM block frame 01 is now occupied by MM block 5.
- 4th CPU request is for MM block 1. CM block frame address is computed as 1 mod 4 =1. But the CM block frame 01 is already occupied by MM block 5. C**onflict miss** occurs **between MM block 1 and MM block 5**
- 5th CPU request is for MM block 3. CM block frame address is computed as 3 mod 4 =3. So CM block frame is now occupied by MM block 3.

Block frame address	00	01	10	11
Block frame content	~~0~~ 3	4	5	1

Fig 2(b): CM in Fig 1(a) is considered as fully associative and it is redrawn in Fig 2(b)

- 1st CPU request is for MM block 0. So CM block frame 00 is now occupied by MM block 0.
- 2nd CPU request is for MM block 4. So CM block frame 01 is now occupied by MM block 4. So no conflict miss occurs in 2nd CPU request like direct mapped.
- 3rd CPU request is for MM block 5. So CM block frame 10 is now occupied by MM block 5.
- 4th CPU request is for MM block 1. CM block frame 11 is now occupied by MM block 1. So no conflict miss occurs in 4th CPU request like direct mapped.
- 5th CPU request is for MM block 3. But all the CM block frames are occupied as shown in Fig 2(b) up to the 4th CPU request in this example. In such a situation **capacity miss** occurs **due to lack of space in CM**. So it is required to select a victim block in CM and replace this victim block by MM block 3.

Set address	Set 0		Set 1	
Block frame address	00	01	10	11
Block frame content	0	4	5	1

Fig 2(c): CM in Fig 2 is redrawn in Fig 2(c)

- 1st CPU request is for MM block 0. Set address is computed as 0 mod 2 =0. So CM block frame 00 in Set 0 is now occupied by MM block 0.
- 2nd CPU request is for MM block 4. Set address is computed as 4 mod 2 =0. The CM block frame 01 in Set 0 is occupied by MM block 4. So no conflict miss occurs in 2nd CPU request like direct mapped.
- 3rd CPU request is for MM block 5. Set address is computed as 5 mod 2 =1. So CM block frame 10 in Set 1 is now occupied by MM block 5.
- 4th CPU request is for MM block 1. Set address is computed as 1 mod 2 =1. The CM block frame 11 in Set 1 is now occupied by MM block 1. So no conflict miss occurs in 4th CPU request like direct mapped.
- 5th CPU request is for MM block 3. Set address is computed as 3 mod 2=1. But both the block frames (10 and 11 in Fig 2(c)) in Set 1 are occupied as shown in Fig 2(c). In such a situation **conflict miss** occurs **between MM block 5, MM block 1 and MM block 3 as all these 3 MM blocks are mapped into the same set.** So it is required to select a victim block in Set 1 of CM among the MM blocks 5 and 1, and replace this victim block by MM block 3.

Associativity

- A direct mapped cache is one-way set associative as a particular MM block can be placed in a particular block frame of CM. Associativity of direct mapped CM is 1.
- Fully associative cache is m-way set associative (m=number of block frames in CM) as a particular MM block can be placed in one of m block frames in CM. Associativity of fully associative CM is m.
- Set associative cache is k-way set associative (k=number of block frames per set in CM) as a particular MM block can be placed in one of k block frames within a particular set. Fig 2 shows a 2-way (k=2) set associative cache.

Address

CPU generates MM address for accessing data during program execution. This address is divided into 2 fields as (Tag, Block offset) in fully associative cache (Fig 3). The tag field is divided into 2 fields as (Tag, Index) both for direct mapped cache and set associative cache (Fig 3).

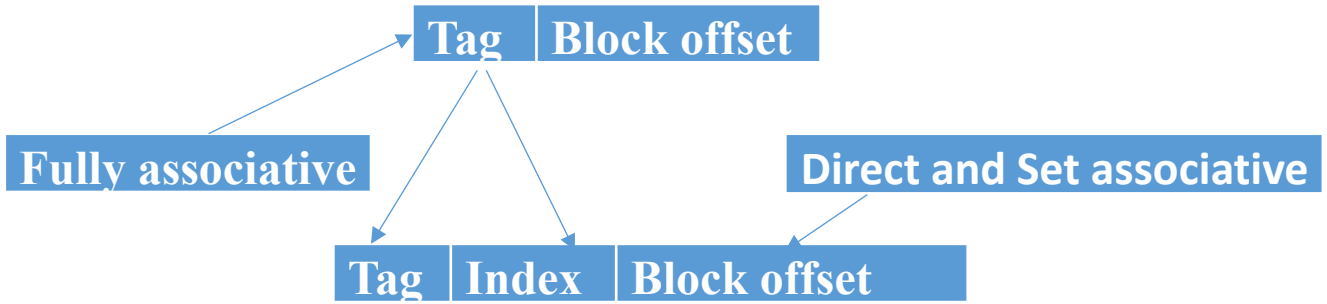


Fig 3

Tag field is used to find hit or miss and block offset field is used to access the desired byte of CPU from a MM block.

The block offset field value remains same (i.e. 1001010110) for fully associative CM, direct mapped CM and set associative CM.

- Index field selects the block frame in CM for direct mapped cache and the set in CM for set associative cache.

For example, let the MM address as generated by CPU is 101 1001010110. So CPU request is generated for byte number 1001010110 from MM block number 101.

- In fully associative mapping Tag=101 and Block offset = 1001010110.
- In direct mapped CM the Tag field 101 is divided as 1 (Tag) and 01 (Index, as index field identifies 1 block frame from 4 block frames in CM so it has 2-bit).
- In 2-way set associative CM (Fig 2) the Tag field 101 is divided as 10 (Tag) and 1 (Index, as index field identifies 1 set from 2 sets in CM (Fig 2) so it has 1-bit).

Example: A fully associative CM has 8 block frames. The least recently used (LRU) policy is used for cache block replacement. The following is the sequence of memory block requests as generated by CPU during program execution.
Sequence→4, 3, 25, 8, 19, 6, **25**, **8**, 16, 35, 45, 22, **8**, 3, **16**, **25**, 7
Initially all the 8 block frames in CM are empty as shown below. Each block frame is identified by 3-bit address (000 to 111) as CM has 8 block frames in this example.

Block frame address	000	001	010	011	100	101	110	111
Block frame content								

Block frame address	000	001	010	011	100	101	110	111
Block frame content	~~4~~ 45	~~3~~ 22	25	8	~~19~~ 3	~~6~~ 7	16	35

- 1st CPU request is for MM block 4. So compulsory miss occurs and CM block frame 000 is now occupied by MM block 4.
- 2nd CPU request is for MM block 3. So again the compulsory miss occurs and CM block frame 001 is now occupied by MM block 3.
- 3rd CPU request is for MM block 25. So again the compulsory miss occurs and CM block frame 010 is now occupied by MM block 25.
- 4th CPU request is for MM block 8. So again the compulsory miss occurs and CM block frame 011 is now occupied by MM block 8.
- 5th CPU request is for MM block 19. So again the compulsory miss occurs and CM block frame 100 is now occupied by MM block 19.
- 6th CPU request is for MM block 6. So again the compulsory miss occurs and CM block frame 101 is now occupied by MM block 6.
- 7th CPU request is for MM block 25. But this block is already available in CM block frame 010 and so hit occurs.
- 8th CPU request is for MM block 8. But this block is already available in CM block frame 011 and so hit occurs .

- 9th CPU request is for MM block 16. So again the compulsory miss occurs and CM block frame 110 is now occupied by MM block 16.
- 10th CPU request is for MM block 35. So again the compulsory miss occurs and CM block frame 111 is now occupied by MM block 35.
- 11th CPU request is for MM block 45. Now all the block frames in CM are occupied. So the **capacity miss** occurs. At this time LRU policy is used to search for a victim MM block in CM. This victim block is a least recently used block i.e. the block corresponding to the oldest CPU request. In this example MM block 4 is such a block as block 4 was the 1st CPU request. So replace block 4 in CM block frame 000 by MM block 45.
- 12th CPU request is for MM block 22. Now all the block frames in CM are occupied. So the **capacity miss** occurs. At this time LRU policy is used to search for a victim MM block in CM. In this example MM block 3 is the next victim block as block 3 was the 2nd CPU request. So replace block 3 in CM block frame 001 by MM block 22.
- 13th CPU request is for MM block 8. But this block is already available in CM block frame 011 and so hit occurs.
- 14th CPU request is for MM block 3. Now all the block frames in CM are occupied. So the **capacity miss** occurs. At this time LRU policy is used to search for a victim MM block in CM. In this example MM block 19 is the next victim block as block 19 was the **5th** CPU request. So replace block 19 in CM block frame 100 by MM block 3.
- 15th CPU request is for MM block 16. But this block is already available in CM block frame 110 and so hit occurs.
- 16th CPU request is for MM block 25. But this block is already available in CM block frame 010 and so hit occurs.
- 17th CPU request is for MM block 7. Now all the block frames in CM are occupied. So the **capacity miss** occurs. At this time LRU policy is used to search for a victim block in CM. In this example MM block 6 is the next victim block as block 6 was the **6th** CPU request whereas the other blocks available in CM block frames at this instant of time are accessed after block 6. So replace block 6 in CM block frame 101 by MM block 7.

Method of deciding hit or miss

Tag=101	Block offset=1001010110

Fig 3 Address

Tag value and block offset value in the address are assumed as 101 and 1001010110 respectively. In fully associative CM as any MM block can be placed in any CM block frame so to decide hit or miss it is required to compare the tag value in the address (i.e. 101) with the tag values that are associated with the CM block frames (shown in Fig 1).

Four 3-bit comparators are required to compare the four tag values that are associated with the four block frames in CM (Fig 1) and the tag value in the address i.e. 101 (Fig 3) in parallel.

Such parallel comparison is required to reduce the searching time.

So in fully associative mapping the number of comparator is equal to the number of block frames in CM and the size of comparator depends upon the size of tag field ($\log_2$(number of blocks in MM)) in the address.

Now consider the output of all the 4 comparators is false i.e. MM block 101 is not available in CM block frames. In such a case miss occurs and the desired byte may be accessed from MM block 101 using the MM address (101 1001010110). The desired byte may also be accessed from CM by identifying a victim MM block in CM and by replacing this victim block by MM block 101 as discussed above.

Hardwire requirement

Number of comparator in fully associative CM is equal to the number of block frames in CM.

Number of comparator in direct mapped cache is 1 as a particular MM block can be placed in a particular block frame of CM.

Number of comparator in k-way set associative CM is k.

Size of comparator depends on the size of tag field in the address.

So the hardwire requirement is high in Fully associative CM than direct mapped CM.

Set associative mapping needs less hardwire than fully associative mapping and more than direct mapped.

Example: A 2-way set associative CM has 4 sets and 8 block frames. MM has 128 blocks. LRU policy is used for cache block replacement. The following is the sequence of memory block requests as generated by CPU during program execution.
Sequence→ 0, 5, 3, 9, 7, **0**, 16, 55.
The number of sets and block frames in CM in this example are 4 and 8 respectively. So the set number or set address contains 2-bit (00 to 11) and the block frame number or block frame address contains 3-bit (000 to 111). Initially all the 8 block frames in CM are empty as shown in the case of fully associative.

Set address	Set 00		Set 01		Set 10		Set 11	
Block frame address	000	001	010	011	100	101	110	111
Block frame content	0	16	5	9			~~3~~–55	7

- 1st CPU request is for MM block 0. So compulsory miss occurs and the set address is computed as 0 mod 4 =0. So CM block frame 000 in Set 00 is now occupied by MM block 0.
- 2nd CPU request is for MM block 5. So compulsory miss occurs and the set address is computed as 5 mod 4 =1. So CM block frame 010 in Set 01 is now occupied by MM block 5.
- 3rd CPU request is for MM block 3. So compulsory miss occurs and the set address is computed as 3 mod 4 =3. So CM block frame 110 in Set 11 is now occupied by MM block 3.
- 4th CPU request is for MM block 9. So compulsory miss occurs and the set address is computed as 9 mod 4 =1. So CM block frame 011 in Set 01 is now occupied by MM block 9.
- 5th CPU request is for MM block 7. So compulsory miss occurs and the set address is computed as 7 mod 4 =3. So CM block frame 111 in Set 11 is now occupied by MM block 7.
- 6th CPU request is for MM block 0. But this block is already available in CM block frame 000 in Set 00 and so hit occurs.

7th CPU request is for MM block 16. So compulsory miss occurs and the set address is computed as 16 mod 4 = 0. So CM block frame 001 in set 00 is now occupied by MM block 16.
8th CPU request is for MM block 55. The set address is computed as 55 mod 4 = 3. But both the block frames in set 11 are occupied. So conflict miss occurs among MM blocks 3, 7, 55. MM block 3 is selected as victim by LRU policy and is replaced by MM block 55.

Performance of CM:

Hit ratio (H): Let N1 and N2 are the number of address referenced (CPU requests for MM access during program execution) satisfied by M1 (level 1 in memory hierarchy i.e. CM) and M2 (level 2 in memory hierarchy i.e. MM) respectively. So H= $\frac{number\ of\ hit\ i.e.the\ number\ of\ address\ referenced\ as\ satisfied\ by\ CM}{total\ umber\ of\ address\ referenced} = \frac{N1}{N1+N2}$. The desirable value of H is close to 1.

Average access time (t_{avg}): Let us consider that **tc** is the access time of CM, **tm** is the access time of MM and 100 is the total number of address referenced as generated by CPU during program execution. Now CM is searched for all the 100 address referenced to find hit or miss. Again consider that hit occurs for 80 address referenced and so miss occurs for 20 address referenced. Hence MM need to be searched for 20 address referenced for which miss occurs. As per the definition of hit ratio $H=\frac{80}{100}$. So $t_{avg}=\frac{100*tc+(100-80)*tm}{100}$ = $tc+(1-\frac{80}{100})*tm=tc+(1-H)*tm$.

[Note: 100*tc is the required time to search CM for 100 address referenced and (100-80)*tm is the required time to search MM for 20 address referenced.]

Example

1. Consider a direct mapped CM of size 32-KB with block size 32-byte. CPU generates 32-bit address. Find the number of bits needed for tag and cache indexing.

Ans. Block size is 32-byte, Size of block offset field is $\log_2(32)$=5-bit (use to access 1-byte from 32-byte of a block)

The number of block frames in CM = $\frac{Size\ of\ CM}{Size\ of\ each\ block\ frame}=\frac{32-KB}{32-byte}=2^{10}$

The number of block frames in CM is 2^{10} and hence 10-bit is required to select a block frame from 2^{10} number of block frames in CM. So the size of index field (index field value determines block frame address in CM) in the address is 10-bit. So the size of tag field is (size of address-size of index field-size of block offset field)=(32-10-5)=17-bit.

2. A 4-way set associative CM consists of 128 block frames. Each block has 64 words. CPU generates an address of size 20-bit for accessing a word from MM. Find the number of bits in the tag, index and block offset field. **[size of block in MM and block frame in CM are identical]**

Ans. Each block has 64 words. Size of block offset field is $\log_2(64)$=6-bit (use to access 1 word from 64 words of a block)
The number of sets in CM = $\frac{Number\ of\ block\ frames\ in\ CM}{Number\ of\ block\ frames\ per\ set\ i.e.\ associativity}$= $\frac{128}{4}$=2^5
The number of sets in CM is 2^5 and hence 5-bit is required to select a set from 2^5 number of sets in CM. So the size of index field (index field value determines set address in CM) in the address is 5-bit.
So the size of tag field is (size of address-size of index field-size of block offset field)=(20-5-6)=9-bit.

[Note: As CM in this problem is set associative type so CM is divided into few sets. Moreover, CM is 4-way set associative. So the number of block frames per set is 4.]

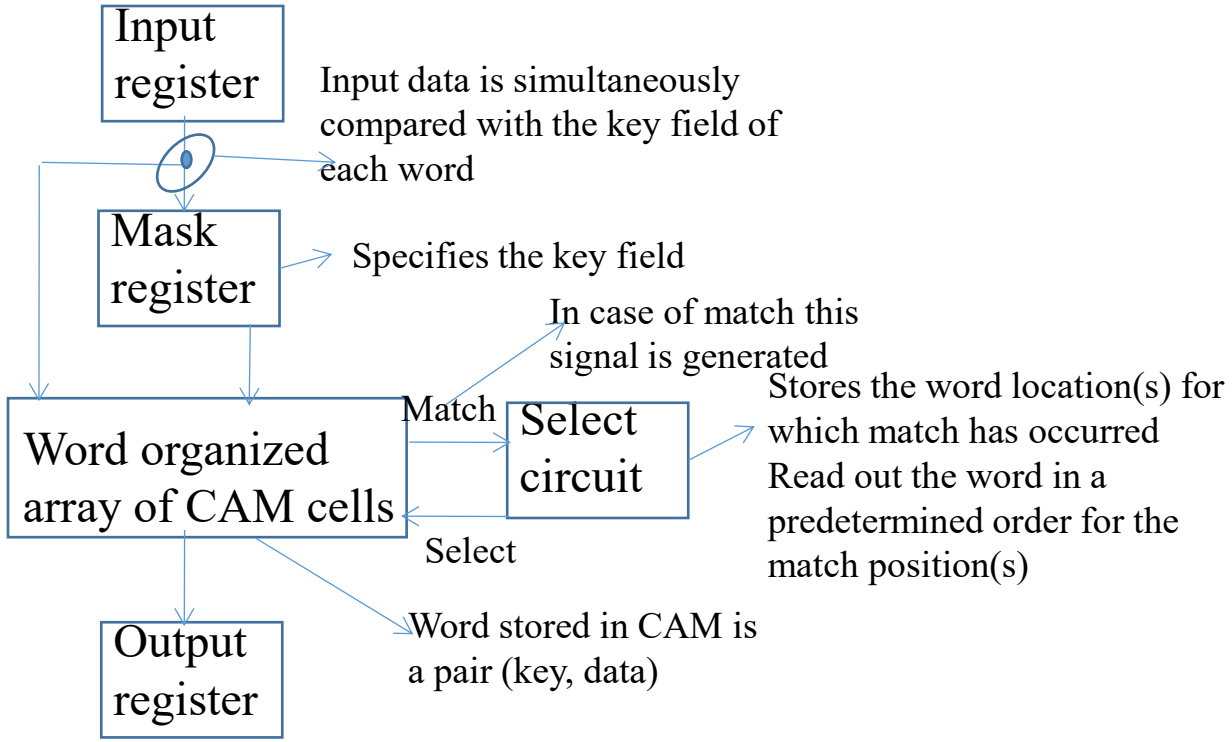
3. The average access time is 140 nsec without level and 30 nsec with level of a two level memory. If the fastest memory access time is 20 nsec, what is the hit ratio?

Ans. The average access time without the level (i.e. the system has only MM no CM) is 140 nsec. So tm=140 nsec.
The average access time with level (i.e. the system has both CM and MM) is 30 nsec. So t_{avg}=30 nsec in presence of both CM and MM.
The fastest memory (CM is faster than MM as discussed in the context of memory hierarchy in the class) access time is 20 nsec. So tc=20 nsec.
Now t_{avg}=tc+(1-H)*tm. So 30=20+(1-H)*140 which gives H=92%

CAM: Content addressable memory is an associative memory
Any stored item can be accessed directly by using the contents of the item in question

Format of item stored in associative memory – Key, Data, key is the address and data is the information to be accessed.

Each unit of stored information is a fixed length word
Key is specified by the mask register and compared simultaneously with all stored words; those which match the key emit a match signal, which enters into a select circuit, select circuit enables the data to be accessed.



If several entries have the same key, then the select circuit determines which data field is to be read out as per some predefined order.
Since all words in the memory are required to compare their keys with the input key simultaneously, each must have its own match circuit.
Advantage over RAM
Capable of performing parallel search and parallel comparison operations
Disadvantage: High hardware cost
As it is required to compare the input key and the key associated with all the words simultaneously each must have its own match circuit
Match and select circuit make CAM much more complex and expensive

Read/Write operation:
Read/Write operation can be performed on each of the words for which match signal is generated
Read or write instruction is preceded by the match instruction having the format (Match key, Input data)

Key Applications of CAM:
Networking Hardware: Used in network routers and switches (Layer 2/3) for incredibly fast MAC address lookup, packet classification, and forwarding decisions.
Cache Memory Systems: Implemented in CPU cache systems to quickly determine if data exists within the cache (tag lookup), enhancing cache speed.
Database Accelerators: Speeds up database management systems by allowing complex, parallel, and rapid search operations.
Pattern Recognition & AI: Utilized in AI, machine learning, and pattern recognition tasks due to its efficient, high-speed data matching capabilities.